

一、先敲定方向：建设“面向仿真任务的知识与执行体系”

我的建议是：以仿真任务为中心，把资料、卡片、模型、案例、操作方法和验收依据关联起来，让大模型能够查依据、选方法、调用工具、检查结果，并从完成和失败的任务中积累经验。

它要同时服务两个目的：

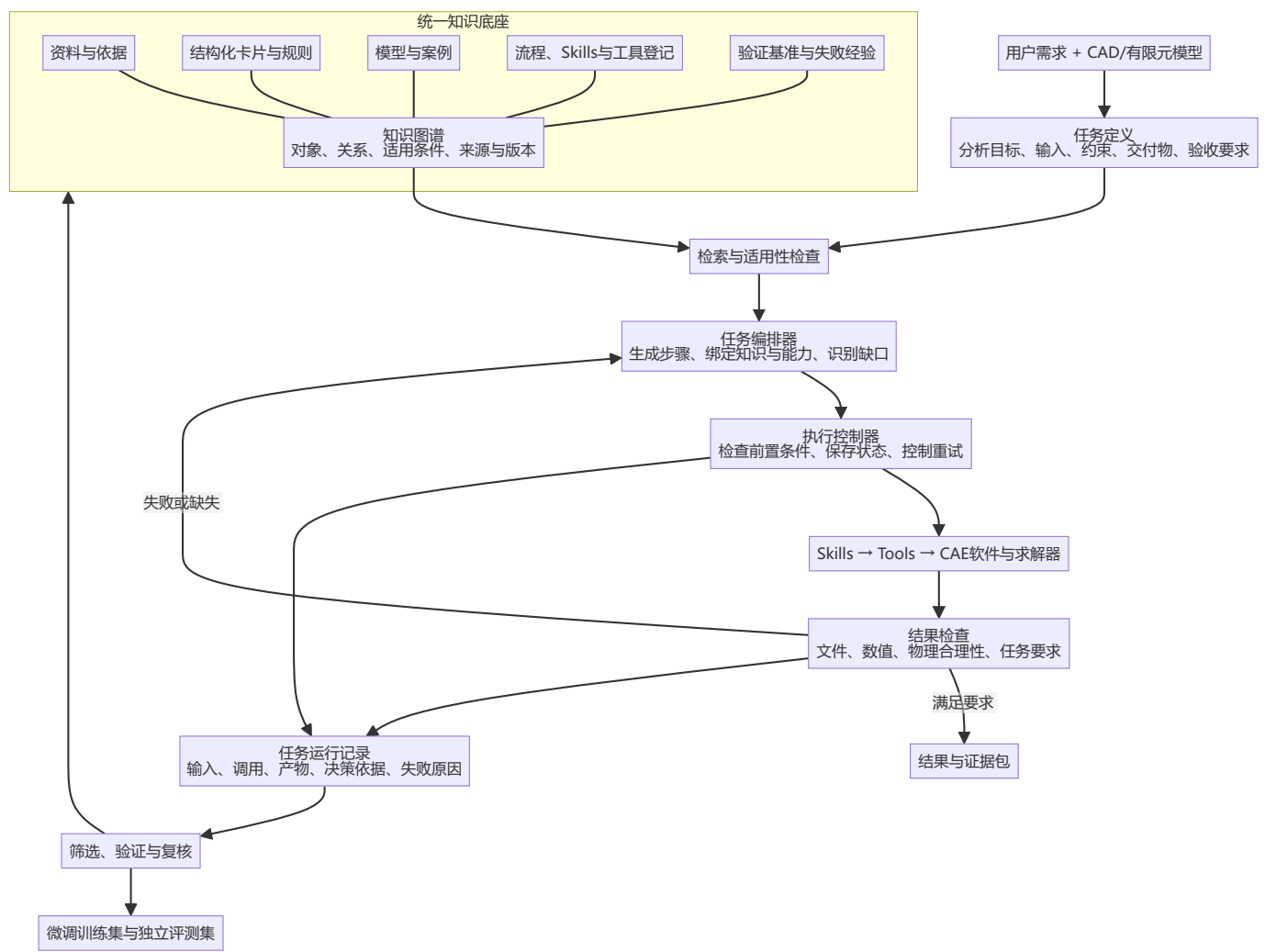
- 1. **执行时使用**：检索当前适用的知识，驱动仿真任务。
- 2. **训练时使用**：从经过验证的任务记录中制作样本，提高大模型的编排、工具调用和失败处理能力。

这里需要调整一个前提：**知识库不能只按照“给大模型微调”来建设**。微调、检索、工具执行和结果验证应共同组成系统。材料参数、载荷依据、模型版本等信息，需要在执行时准确获取，不能依靠模型记忆。

同样，知识库不能保证所有仿真任务成功；合理的目标是：**提高支持范围内任务的完成率，准确识别缺口，避免把错误结果当作完成**。

下面以你们当前的结构仿真为主线，给出建议架构。涉及原有“四类卡片、原始资料可切片而卡片不切片”的安排，沿用此前讨论记录；本次没有重新审计现有软件实现。

二、总体架构：知识底座、任务编排、工具执行、经验回流



这个架构最重要的约束是：

每一步都要能回答：为什么做、依据什么、输入是否齐全、用哪个能力执行、输出是什么、怎样判定完成、失败后怎么办。

大模型负责理解、选择和提出计划；执行控制器根据真实工具返回与检查结果推进任务状态。

三、从“任务成功需要什么”反推知识库内容

资料、案例、卡片、数模都应该保留，但还需要补齐**流程知识、能力知识、验证知识和失败知识**。

下表是逻辑分类，不意味着必须部署九套数据库。

知识类别	应该包含什么	建议保存形式	对任务完成的作用
1. 资料与依据	理论、方法说明、软件手册、材料来源、试验报告、工程要求	原文件 + 带页码/章节定位的解析内容	回答“为什么这样做，依据在哪里”
2. 结构化卡片	材料、术语、网格策略、工况	完整结构化记录，数值和单位分开保存	让模型准确选取和读取知识
3. 工程模型资产	CAD、装配、有限元模型、网格、输入文件、结果文件	原生文件 + 资产清单 + 语义描述	提供实际操作对象和可复用模型
4. 仿真案例	目标、模型、假设、参数依据、步骤、工具版本、结果、检查记录	案例说明 + 引用各资产的清单	回答“类似任务做过什么，哪些可以复用”
5. 流程与决策规则	分析类型选择、步骤依赖、输入要求、分支条件、停止条件	流程模板 + 结构化规则	指导任务拆解和顺序安排
6. Skills	可复用操作方法、适用范围、前置条件、工具组合、验收和恢复方法	Skill说明 + 机器可读描述 + 脚本/模板	回答“这一步具体怎么做”
7. 工具能力登记	接口、参数、返回结构、支持格式、软件版本、运行环境、可用状态	接口定义 + 工具登记记录	回答“现在到底能调用什么”
8. 验证与基准	解析解、试验对照、已复核算例、收敛方法、检查规则及容差依据	基准文件 + 检查程序 + 预期结果	回答“怎样知道做对了”
9. 失败与恢复经验	错误现象、诊断证据、原因、适用修复、失败修复、停止条件	结构化失败记录 + 关联运行日志	减少重复试错，支持有依据的恢复

四类卡片如何放在这里？

建议继续保留你们已有的四类：

- **材料卡**：材料本体参数、温度/方向/状态适用范围、来源。
- **术语卡**：概念定义、别名、单位含义、容易混淆的边界。
- **网格策略卡**：适用几何、单元选择、网格控制方法、质量与收敛要求。
- **工况卡**：分析意图、载荷类型、约束含义、适用层级、所需上游输入。

需要特别明确：

工况卡可以告诉系统“需要什么载荷”，但未必已经包含当前模型可直接施加的载荷值和作用位置。

例如部件受力来自整机分析时，必须关联上游载荷结果及其传递方法。不能找到名称相似的工况卡，就直接生成一组载荷。

同理，材料卡保存材料属性；厚度、截面、铺层等属于具体模型。沿用此前约定，材料导出保持在 MAT1/MAT8 范围，PSHELL/PCOMP 等由模型属性确定。

四、知识实际怎么存：原件保留，结构化管理，图谱关联，索引检索

1. 推荐的存储分工

存储部分	放什么	关键要求
文件/对象存储	PDF、CAD、BDF、OP2、图片、日志、报告等原件	有稳定资产ID、版本、校验值和位置
结构化主库	卡片、模型元数据、案例清单、规则、Skill和工具登记	字段明确，支持精确查询与版本管理
图谱存储	对象之间的关系、适用条件、依据关系、版本依赖	能沿关系找到相关知识和受影响对象
全文与向量索引	文档片段、案例摘要、卡片摘要、Skill能力描述	用来发现候选，命中后回读完整记录
运行与数据集存储	每次任务状态、工具调用、检查结果、训练样本	任务记录与长期有效知识分开管理

同一份知识要有明确的权威记录。图谱和搜索索引可以引用或派生这些记录，不应各自维护一份互相矛盾的材料参数。

初期可以先用现有文件系统和结构化数据库承载主体，再接入图谱与检索组件，不必先建设庞大的分布式平台。

2. 不同内容采用不同处理方式

- 资料：按章节、语义段落处理；表格保留表头、单位、脚注和页码。
- 卡片：作为完整结构化对象保存；可以给摘要建向量索引，读取时返回整张卡。
- 案例：拆出目标、模型、步骤、依据、结果等字段，同时保留完整案例包。
- CAD/有限元文件：通过相应解析工具提取几何和模型元数据，原文件保留。
- Skills：检索时先读取能力摘要，选中后再加载操作说明和工具定义。
- 运行记录：按任务和步骤保存，经过筛选才能进入可复用案例或训练集。

3. 每个重要知识对象都需要共同的“身份信息”

至少包括：

ID、类型、版本、来源定位、适用范围、状态、关联对象、验证依据。

其中来源最好做到**字段级**：同一材料卡中的弹性模量、密度和强度可能来自不同来源，不能只给整张卡挂一个文件名。

知识也需要区分：

- 来源明确的事实；
- 经过适用性检查的可用知识；
- 自动抽取的候选；
- 推导或假设；
- 存在冲突、失效或待确认的内容。

这些状态不能仅由模型自报的“置信度”决定。

五、图谱怎么建：连接“对象、依据和操作”

1. 先定义仿真领域的对象与关系

这里的“本体”，可以直接理解为：**统一规定系统里有哪些对象、对象有什么属性、允许哪些关系。**

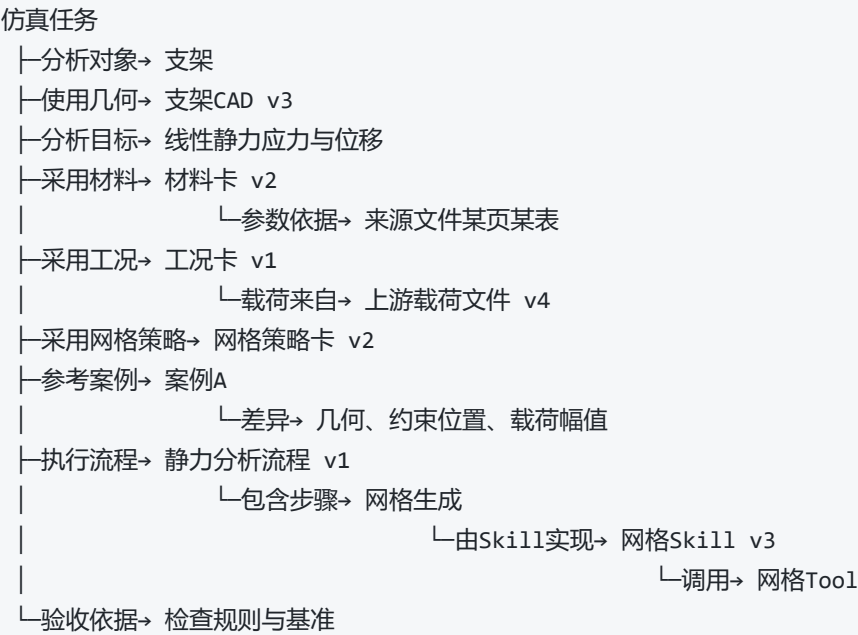
建议首批对象包括：

仿真任务、分析目标、零件/装配、几何版本、有限元模型、材料、工况、载荷来源、网格策略、案例、流程、Skill、Tool、检查规则、运行记录和结果。

不要一开始从所有资料中无约束地抽取海量关系。先围绕需要完成的任务定义关系，再补内容。

2. 一个具体关联示例

以下是示意关系，不代表已经存在的模型或案例：



这样系统就能回答：

- 这张材料卡适用于当前温度和材料状态吗？
- 这个案例的边界条件与本任务有哪些差异？

- 当前步骤有哪些可用Skill？ 分别依赖哪些工具？
- 某份载荷更新后， 哪些任务结果需要重新计算？
- 这次结果到底使用了哪些版本的依据？

相似关系只帮助发现候选；适用关系才支持实际采用。

3. 模型资产还要建立“语义—几何—网格”的映射

例如：

“安装孔内壁”

- 当前CAD版本中的面集合
- 当前网格版本中的节点/单元集合
- 约束或载荷的作用区域

这是从自然语言走向仿真执行的关键。

不能长期依赖 “第12号面” “节点1001” 这样的编号。几何或网格变化后，映射应重新解析并检查，旧结果不能直接沿用。

六、如何高效检索知识图谱和Skills？

建议采用：任务结构化 → 条件筛选 → 混合检索 → 有限图谱扩展 → 适用性检查 → 按步骤装配上下文。

第一步：先明确要完成什么

把用户需求整理为一份“任务定义”：

对象：什么零件/装配，哪个模型版本
目标：静力、模态、屈曲等，要求输出什么
条件：材料、载荷、约束、环境、单位
资源：可用软件、工具、计算资源
验收：要求哪些文件、检查和精度证据
缺口：哪些信息还没有依据

其中未知字段保留未知，不能通过检索“猜成已知”。

第二步：针对问题路由到对应知识

当前问题	优先检索
某概念是什么意思	术语卡 + 原文依据
应使用哪组材料参数	材料结构化字段 + 适用条件 + 来源
类似任务怎么完成	案例 + 流程 + 差异说明
当前步骤怎么执行	Skill登记 + 工具能力

当前问题	优先检索
某参数为什么这样取	图谱依据路径 + 来源原文
工具执行失败怎么处理	错误代码/日志特征 + 失败案例

不需要每个问题都遍历整个图谱。

第三步：先筛掉不适用项，再排序

对执行候选，先检查分析类型、材料状态、软件版本、输入格式等硬条件；未知条件单独标记。

再结合：

- 关键词检索；
- 语义相似检索；
- 图谱关系；
- 已验证案例；
- 本任务适用性；

进行排序。不能让高相似度抵消明确的不适用条件。

第四步：只取当前步骤需要的关系和内容

例如网格阶段优先读取几何特征、网格策略、相关案例、网格Skill及其检查项。

图谱按规定关系有限扩展；可以从一到两跳开始，对必要依赖继续受控展开。控制返回数量、耗时和上下文长度。

第五步：返回“可执行知识包”

检索结果应包括：

推荐对象及版本、适用理由、依据定位、所需输入、可用Skill/Tool、检查要求，以及仍未解决的缺口。

检索效率也来自复用：可缓存稳定查询结果，但缓存必须关联知识、模型、工具和索引版本，变化后失效。

七、Skill与Tool如何联动？没找到合适的怎么办？

1. 三者分工

- **流程**：安排整项任务的步骤与依赖。
- **Skill**：描述某个步骤如何可靠完成。
- **Tool**：执行具体操作，例如读取CAD、生成网格、提交求解、解析结果。

一个Skill可以调用多个Tool；同一个Tool也可以被多个Skill复用。

建议每个Skill至少声明：

字段	作用
能完成的目标	用于检索与选择
适用范围、禁用条件	避免误用
输入及前置条件	判断当前能否执行
输出及完成条件	判断目标是否实现
工具和版本依赖	判断环境是否支持
参数获取规则	明确来自用户、测量、卡片还是上游结果
检查项	验证产物和行为
常见失败、恢复与停止条件	控制重试
验证案例	证明该Skill在哪些范围内有效

只有描述“我会做网格”不够；必须明确给什么输入、产出什么，以及怎么验收。

执行时还要检查相邻步骤是否衔接：输出格式、单位、坐标系、实体标识和版本必须兼容。

2. 检索不到时，按缺口类型处理

缺口	建议行为
名称没匹配上	用术语卡扩展别名、英文名、软件术语后再检索
没有完整流程	尝试组合已验证的步骤级Skills，并检查依赖和输入输出
只有相近案例	提取可复用方法，明确差异；重新获得本任务参数
缺材料、载荷或边界条件	追溯来源，使用已验证工具测量/计算，或请求必要输入
缺某项Skill，但工具齐全	生成候选Skill，在隔离算例中验证后再使用
缺Tool或环境不支持	查登记的替代能力；无法满足时输出具体能力缺口
检索系统故障或索引滞后	标记检索异常，不能报告“知识不存在”
多轮修复仍失败	根据预算与停止条件终止，保留失败证据和后续需求

这里建议加入一种正式对象：“知识/能力缺口记录”。

它保存“哪个任务卡在什么地方、缺什么、搜索过什么、补齐后怎样验收”。这样后续资料建设有明确优先级。

系统可以自动形成新知识和新Skill的候选，但进入正式可用集合，需要相应检查。一次跑通不能自动推广为所有同类任务都适用。

八、如何驱动仿真任务，又如何用于微调？

1. 执行时：让知识真正进入参数与操作

以“导入支架STEP，完成静力分析并输出位移和应力”为例：

- 1. **解析模型**：识别单位、几何和候选作用区域。
- 2. **补齐任务定义**：确认材料、载荷、约束依据及输出要求。
- 3. **检索并检查适用性**：选择卡片、案例、流程和Skills。
- 4. **绑定到本模型**：把语义区域映射到当前几何及网格。
- 5. **执行前检查**：输入完整、工具可用、单位与坐标一致。
- 6. **网格、建模与求解**：各阶段保存实际产物和状态。
- 7. **检查结果**：求解状态、结果完整性，以及按任务适用的平衡、变形、收敛或基准检查。
- 8. **交付证据包**：模型、输入、日志、结果、可视化和检查说明。

对采用Nastran的流程，应绑定实际BDF、F06、OP2及结果显示证据；其他求解器使用相应产物。

求解器退出成功、生成结果文件、完成工程验收，应分别记录。 检查容差也需要来源和适用条件，不能让模型现场编一个阈值。

2. 微调时：重点教“怎么完成任务”

适合训练的能力	样本应体现什么
理解需求	自然语言如何转成任务定义
拆解与编排	当前条件下如何选择步骤和依赖
检索与选用	为什么选择某条知识，排除其他候选
工具调用	正确参数结构、调用顺序、返回解释
缺口识别	什么时候可以继续，什么时候必须补信息
故障恢复	错误观察、诊断依据、修复操作与结果
完成判断	如何依据真实产物与检查结果宣布完成

材料具体数值、当前文件路径、模型编号和软件实时状态，应主要通过检索和工具取得。

一个训练样本可以组织为：

任务目标与当前状态

+ 当时可见的知识及版本

+ 允许使用的工具定义

→正确的下一步动作

+ 真实工具返回

+ 检查结果

+ 简明选择依据

训练目标来自**可观察、可验证的行为记录**，无需保存模型隐藏思考过程。

3. 经验回流需要独立验证

推荐顺序：

真实运行记录 → 筛选与纠错 → 可复用案例/Skill候选 → 验证 → 训练集 → 独立评测 → 决定是否采用新模型。

训练集和评测集要按案例来源、模型家族等划分，避免同一个模型的小改版同时落入两边，导致成绩虚高。

优先建立检索与执行基线，再看瓶颈：

- 知识存在但查不到：改检索。
- 参数来源不完整：补知识。
- 工具不稳定：修工具。
- 有正确知识与可用工具仍不会编排：重点考虑微调。

九、Semantica怎么用：作为语义与图谱层候选

我查看了 [Semantica GitHub仓库](#)、架构说明和模块文档。它是独立开源项目；“开源Palantir”适合作为定位类比，不能理解为Palantir官方开源版或功能等价替代。

Palantir值得借鉴的思想是：把**对象、属性、关系与动作、函数**连接起来，使数据能够支持实际操作。[Palantir Ontology官方说明](#)

Semantica文档描述了本体与图谱、混合检索、来源追溯、决策记录和接口接入等能力，因此适合放在本方案的知识服务层。[Semantica仓库](#)

可借用或验证的部分	我们仍需要建设的部分
对象、关系与本体管理	仿真领域对象、字段和适用规则
图谱辅助检索	面向任务阶段的检索路由与选用检查
来源和决策记录	字段级工程依据、任务版本与产物绑定
图结构约束检查	单位、边界条件、数值和物理合理性检查
知识服务接口	CAD、网格、求解器、结果查看工具
通用上下文管理	执行状态、重试、恢复和验收控制

例如其本体模块提供SHACL图结构约束检查，适合检查必填属性和关系形状；是否满足物理要求，仍需要我们自己的检查程序。[Semantica本体模块](#)

它也描述了图谱辅助检索和历史决策检索，可用于查找关联知识与参考决策。[Semantica上下文模块](#)

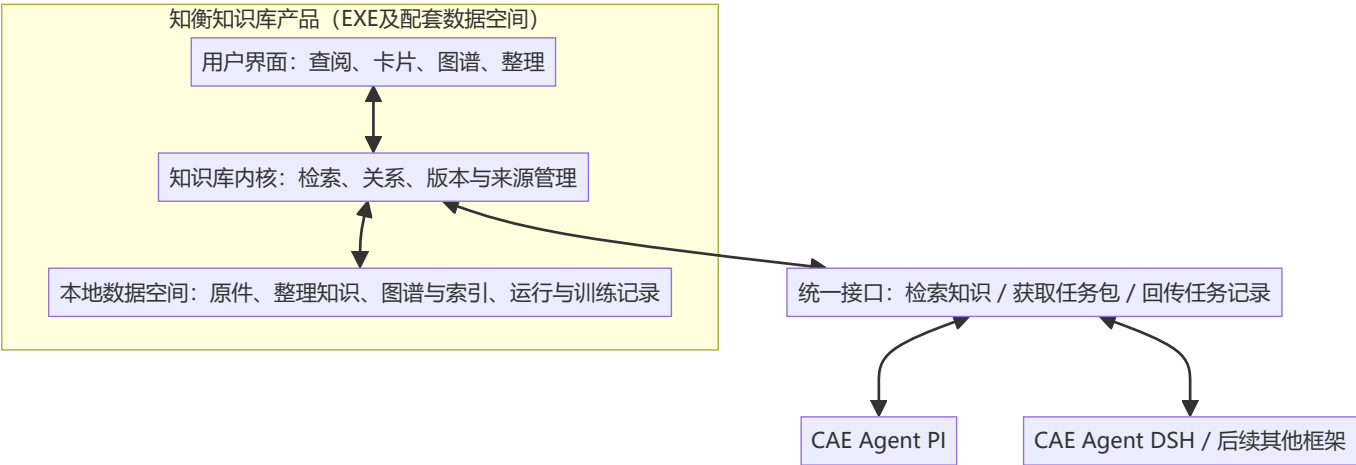
我的建议：架构上采纳这些思想，组件上先做小范围验证。 验证重点是中文工程资料、完整卡片读取、带条件的关系查询、版本更新后的检索一致性，以及任务依据是否能完整追溯。本次属于仓库与文档调研，尚未完成本地运行验证。

智能体内核同样通过统一知识接口和工具接口接入。先定义任务与能力契约，再用同一批仿真任务评测候选内核。

十、最终交付：独立使用的知衡知识库 EXE

最终计划交付一套可独立安装和使用的桌面知识库。用户打开 EXE，就能检索和阅读原始资料、查看知识卡片与模型案例、浏览知识图谱、整理知识和准备任务知识包，无需先启动 CAE Agent。当前 HTML 是交互设计原型，尚未连接真实数据库或智能体。

1. 一个知识库产品，包含界面、内核和数据空间



知识库内核就是这个产品内部负责管理和查询知识的程序。 它随桌面软件安装，由软件自动启动和管理；不要求用户另外操作一套“独立知识服务”。“独立”主要指知识库不依赖 PI 或 DSH，换智能体框架时仍可使用同一套知识。

数据保存在知识库管理的磁盘目录和数据库中，不塞进 EXE 文件本身。 EXE 是程序入口；配套数据空间才保存资料 and 知识。原始资料与整理后的内容都保留。默认导入时复制原件；如采用外部文件引用，必须标明原件位置和可用状态。迁移或备份时要包含数据空间，不能只复制 EXE。

2. 知识库里究竟保存什么？

层次	保存内容	用途与边界
原件层	PDF、CAD、BDF、报告、图片、求解结果文件等	保留原始依据和资产；记录来源、版本、路径和校验值。原件入库不等于内容已验证或适用于当前任务。
整理层	材料卡、术语卡、网格策略卡、工况卡；模型信息、案例、Skill、规则与版本	保存可查询的完整知识对象、适用条件和原文定位；区分候选、已复核等状态。Skill 保存方法、输入输出与版本，执行环境由 Agent 管理。
检索层	全文索引、向量索引、知识图谱	帮助找资料和关联对象；由原件及整理知识生成，可重新构建。图谱不是另一份独立的工程参数权威库。

此外，知识库保留任务知识包快照、运行日志、产物引用、检查与复核记录，以及经筛选冻结的训练 / 评测数据集。未经复核的任务记录保留在候选区，不自动变为可直接采用的有效知识。

界面按“资料与依据、知识卡片、模型与案例、知识图谱、流程与能力、任务知识包、运行与验证、导入与质量管理”等入口组织。图谱采用类似 Obsidian 的节点连线浏览方式，支持缩放、拖动、筛选、局部关系和点击查看来源。

3. CAE Agent 如何使用，又如何积累新知识？

1. **先检索。** PI 或 DSH 通过统一接口提交任务目标、分析类型、模型和工况条件。知识库返回相关资料、完整卡片、模型案例、方法与检查规则，并携带来源、版本和适用范围。
2. **形成任务知识包。** 锁定本任务采用的知识版本，检查必需输入和适用条件。缺失或冲突应明确返回，由 Agent 补齐或暂停相关步骤，不用相似案例中的数值擅自代填。
3. **执行并记录。** Agent 负责调用建模、网格、求解和后处理工具；按任务保存输入、工具调用、日志、结果文件、检查结果与报告。知识库负责提供和管理知识，CAE 执行由 Agent 完成。
4. **回传并整理。** 任务结束后，Agent 通过接口回传，或导出任务包后由用户导入。成功案例、失败原因、恢复过程和经验建议都先登记为候选，并关联实际运行与产物。
5. **复核后再入有效知识。** 核对来源、产物完整性、适用条件和检查结果；需要工程判断的内容由人员复核。通过后形成新版案例、经验卡或 Skill；未通过的保留状态和原因。
6. **下一次继续使用。** 更新索引和图谱，后续任务可检索到这些经过整理的知识。训练数据另行筛选与评测，不把全部运行日志自动用于微调。

知识库提供依据 → Agent 检索并执行 → 回传任务记录与产物 → 整理和复核经验 / 案例 → 版本化入库 → 支持下一次任务。 工具运行成功、自动检查通过和人工工程复核分别记录，不能互相替代。

4. 最终交付包含哪些部分？

交付内容包括：桌面 EXE 及运行依赖、自动管理的知识库内核、本地数据空间及已整理内容、PI / DSH 统一接口与接入说明，以及导入、导出、备份和恢复功能。两类 Agent 各自独立开发；知识库不随 Harness 更换而重建。界面原型用于确认设计，正式交付还需验证实际检索、原件打开、图谱关联、Agent 接入和经验回流。